Ambientes de desenvolvimento

Higor Alexandre Duarte Mascarenhas

Google Colab

- A Google disponibiliza uma plataforma para desenvolvimento Python, muito similar ao Jupyter notebook/lab que o desenvolvedor pode instalar em sua máquina pessoal.

- Uma customização do Jupyter, porém necessita de internet disponível a todo instante no momento de uso.

Google Colab

- Vantagens:
  - Não precisa ter o Anaconda/Python/Jupyter instalado;
  - Não precisa ficar gerenciando bibliotecas;
  - Acesso a GPUs e TPUs gratuitas (use se necessário).
  - Acesso de qualquer lugar (até mesmo de outros dispositivos), todo o conteúdo fica na nuvem.
  - Compartilhamento fácil.

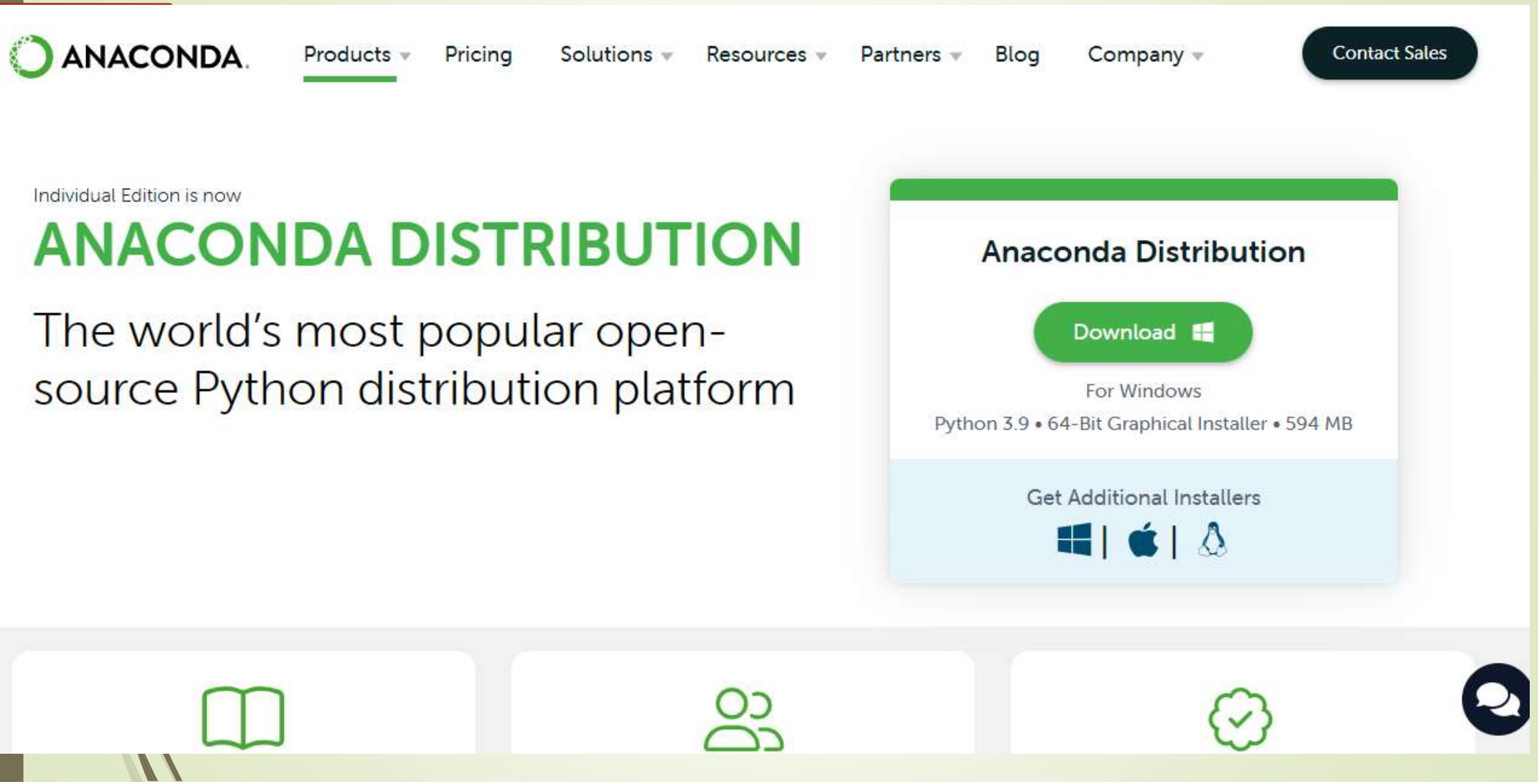
Google Colab

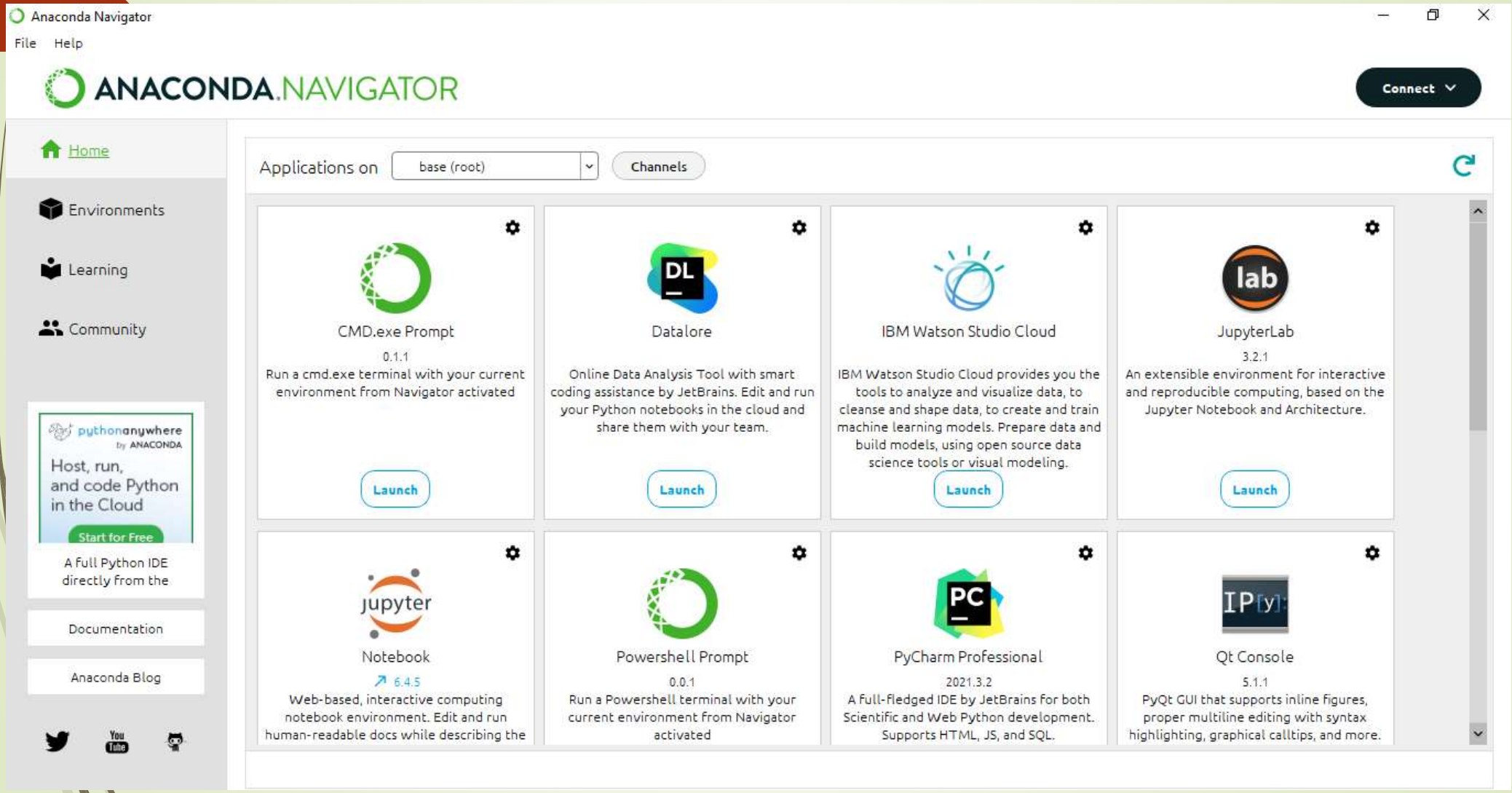
- Não depende de uma máquina local.
- Desvantagens:
  - Execução interrompida após algumas horas (12 horas), versão pró 24 horas.
  - Bibliotecas não disponíveis precisam ser reinstaladas a cada execução.

Instalação do Python na máquina

- Utilizaremos inicialmente o Jupyter notebook.

- A instalação é possível ser efetuada a partir do site:
- https://www.anaconda.com


ANACONDA.
Products
Pricing
Solutions
Resources
Partners
Blog
Company
Contact Sales
Individual Edition is now
ANACONDA DISTRIBUTION
The world's most popular open-source Python distribution platform
Anaconda Distribution
Download
For Windows
Python 3.9 • 64-Bit Graphical Installer • 594 MB
Get Additional Installers


Anaconda Navigator
File Help
ANACONDA.NAVIGATOR
Connect
Home
Environments
Learning
Community
Applications on
base (root)
Channels
CMD.exe Prompt
0.1.1
Run a cmd.exe terminal with your current environment from Navigator activated
Launch
Datalore
Online Data Analysis Tool with smart coding assistance by JetBrains. Edit and run your Python notebooks in the cloud and share them with your team.
Launch
IBM Watson Studio Cloud
IBM Watson Studio Cloud provides you the tools to analyze and visualize data, to cleanse and shape data, to create and train machine learning models. Prepare data and build models, using open source data science tools or visual modeling.
Launch
JupyterLab
3.2.1
An extensible environment for interactive and reproducible computing, based on the Jupyter Notebook and Architecture.
Launch
pythonanywhere by ANACONDA
Host, run, and code Python in the Cloud
Start for Free
A full Python IDE directly from the
Documentation
Anaconda Blog
Notebook
6.4.5
Web-based, interactive computing notebook environment. Edit and run human-readable docs while describing the
Powershell Prompt
0.0.1
Run a Powershell terminal with your current environment from Navigator activated
PyCharm Professional
2021.3.2
A full-fledged IDE by JetBrains for both Scientific and Web Python development. Supports HTML, JS, and SQL.
Qt Console
5.1.1
PyQt GUI that supports inline figures, proper multiline editing with syntax highlighting, graphical calltips, and more.

Ipython

- É um shell para o Python (e também para outras linguagens) com muito mais funcionalidades que o shell padrão do Python.
- Permite copiar e rodar scripts de pequenos códigos.
- Tem uma particularidade de criar pequenos cadernos (notebooks), possibilitando ir registrando as atividades como se fosse um diário.

Jupyter notebook

- Aplicativo Open source para desenvolvimento.
- Ferramenta IDE usada para escrever código em Python e documentar código.
- Gera arquivos IPythonNotebook (.ipynb), arquivo específico para abrir no jupyter notebook.
- Também pode ser utilizado para gerar arquivos .py.
- Excelente alternativa para desenvolvimento e prática de programação em Python em praticamente todas as fases de atividades do profissional de Data Science

Jupyter lab

- É a próxima geração do Jupyter notebook.
- Open source, é a renovação do Jupyter Notebook, que possibilita a utilização de abas, instalação de extensões para os mais variados fins: #github, #spreedsheet(visualização de planilhas), entre outros.
- Ótima alternativa para desenvolvedores para estudar Data Science.
- Também pode ser utilizado para gerar arquivos .py.

Jupyter notebook – conceitos básicos

- Markdown -> Utilizada para documentar e escrever textos;
- Code -> Utilizada para códigos
  - Utiliza-se o símbolo # para descrever o texto

Comandos básicos:

- Shift + Enter -> Rodar célula e ir para próxima
- Ctrl + Enter -> Rodar célula
- Alt + Enter -> Insere uma célula abaixo

Conceitos básicos - Python

- Python -> Tipagem forte(diferencia letras maiúsculas de minúsculas)
- Tipos de variáveis mais comuns:
  - String
  - Float
  - Int
  - Bool
  - Lista
  - Dicionário

Operadores- Python

- Operadores aritméticos
  - + → Soma (2+3 = 5)
  - - → Subtração (3 - 1 = 2)
  - * → Multiplicação (2 * 2 = 4)
  - / → Divisão (2 / 2 = 1)
  - ^ → Potência (2 ^3 = 8)
- Operações com texto:
  - + → Soma ('Oi' + '!' = Oi!)
  - * → Multiplicação ('Oi' * 2 = OiOi)
- Operadores lógicos:
  - and, or , not

Comparação - Python

- == → Igual
- != → Diferente
- > → Maior
- >= → Maior ou igual
- < → Menor
- <= → Menor ou igual

Nomes válidos e inválidos de variáveis

Nomes válidos de variáveis	Nomes inválidos de variáveis
balance	current-balance (hifens não são permitidos)
currentBalance	current balance (espaços não são permitidos)
current_balance	4account (não pode começar com um número)
_spam	42 (não pode começar com um número)
SPAM	total_$um (caracteres especiais como $ não são permitidos)
account4	'hello' (caracteres especiais como ' não são permitidos)

Conceitos básicos - Funções básicas e métodos

- Funções básicas:
  - print() → Imprime o texto ou variável entre parêntese na tela
  - import → Carrega bibliotecas Python
  - !pip install → instala bibliotecas Python
- Métodos - funções implícitas dentro de cada tipo de variável, acessadas usando ponto após a variável (ex: string.replace → Substitui uma parte do texto por outra)
- Instruções sobre uso de funções e métodos → Shift + Tab no Jupyter Notebook

Listas

- Usadas para armazenar vários valores dentro de uma mesma variável
- Identificadas por colchetes (Ex: lista = [1, 2, 3, 'olá', 0.5])
- Operações com listas:

  lista 1 + lista 2 = Lista contendo todos os elementos da lista 1 e lista 2

  lista 1 * 2 = Lista com informações da lista 1 repetidos 2 vezes

Listas – Principais métodos

- lista.append(n) → Adiciona o elemento n à lista
- lista.remove(n) → Remove o elemento n da lista
- lista.sort() → Ordena a lista
- lista.count(n) → Conta quantos elementos n na lista
- lista.len() → Conta quantos elementos na lista
- lista.index(n) → Retorna qual a posição do elemento n na lista

Dicionários

- Dicionários são usados para armazenar pares chave-valor, para posterior uso ou comparação
- Identificados por chaves
- Muito usados para realizar de-para de chaves-valor.

dict = { ‘Segunda’ : 1, ‘Terça’:2, ‘Quarta’:3 , ‘Quinta’: 4 , ‘Sexta’: 5}

Dicionários – Principais métodos

- dict.values() → Retorna os valores armazenados no dicionário
- dict.get(n) → Retorna o valor associado à chave n no dicionário
- dict.pop(n) → Remove a chave n e o valor associado a ela no dicionário
- dict.keys() → Retorna as chaves armazenadas no dicionário

Bibliotecas Python essenciais

- NumPy: Base do processamento numérico em Python;
- O NumPy contém, entre outras coisas:
  - Objeto de array multidimensional ndarray rápido e eficiente.
  - Funções para o processamento de todos os elementos de um array ou para a execução de operações matemáticas entre arrays.
  - Ferramentas para a leitura e a gravação em disco de conjuntos de dados baseados em arrays.
  - Operações de álgebra linear, transformadas de Fourier e geração de números aleatórios.
  - Uma API C madura para permitir que as extensões do Python e códigos nativos C or C++ acessem estruturas de dados e recursos computacionais do NumPy.

Bibliotecas Python essenciais

- **Pandas**: Fornece estruturas de dados e funções de alto nível projetadas para tornar intuitivo e flexível o trabalho com dados estruturados ou tabulares.
- **Matplotlib**: é a biblioteca Python mais popular para a produção de plotagens e outras visualizações de dados bidimensionais
- **SciPy**: é uma coleção de pacotes que resolvem vários problemas básicos de computação científica.
- **Scikit-learn**: desde a criação, tornou-se o principal kit de ferramentas de uso geral para *machine learning* dos programadores Python.

Variáveis e passagens de argumentos

- Quando fazemos uma atribuição a uma variável (ou a um nome) em Python, mos uma referência ao objeto exibido no lado direito do sinal de igualdade uma abordagem mais prática, considere uma lista de inteiros:

a = [1, 2, 3]

- Ao atribuir a a uma variável b,
- b = a, o valor de b será o mesmo de a.
- Em algumas linguagens, a atribuição a **b** faria os dados [1, 2, 3] serem copiados. Em Python, na verdade agora **a** e **b** referenciam o mesmo objeto, a lista original [1, 2, 3].

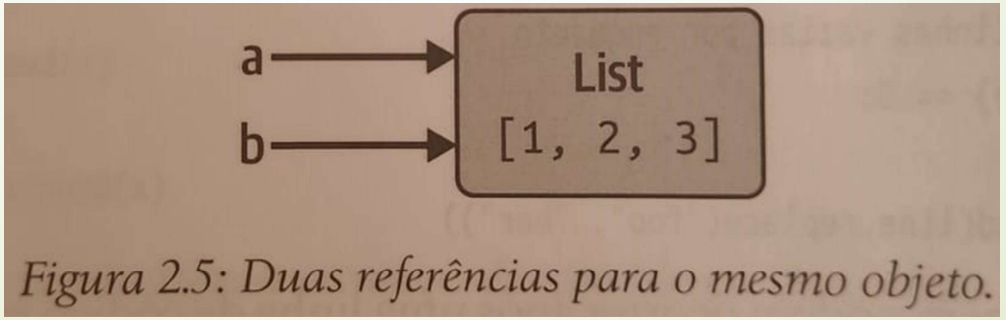


Figura 2.5: Duas referências para o mesmo objeto.

Variáveis e passagens de argumentos

- Ao acrescentar um elemento a a e ao exeminar b, teremos como resultado:

```
In [11]: a.append(4)

In [12]: b
Out[12]: [1, 2, 3, 4]
```